

NVFLARE 2.6.2 — Events Guide

Readable reference for EventType & AppEventType, S&G; sequence, and timing windows.
Updated 2025-09-13 20:32

1) All Events (with concise explanations)

System Events — EventType

Client Connectivity & Register

- CLIENT_DISCONNECTED — Server detected a client disconnect.
- CLIENT_HEARTBEAT_PROCESSED — Server processed a client heartbeat.
- CLIENT_HEARTBEAT_RECEIVED — Server received a client heartbeat.
- CLIENT_QUIT — Client terminated the run voluntarily.
- CLIENT_RECONNECTED — System coordination event.
- CLIENT_REGISTER_PROCESSED — Server processed a client registration.
- CLIENT_REGISTER_RECEIVED — Server received a client registration request.

Job Lifecycle & Deployment

- DEPLOY_JOB_TO_CLIENT — Deploy job to client sites.
- DEPLOY_JOB_TO_SERVER — Deploy job on server.
- JOB_ABORTED — Job aborted due to error or request.
- JOB_CANCELLED — Job cancelled.
- JOB_COMPLETED — Job finished successfully.
- JOB_DEPLOYED — Job has been deployed to target(s).
- JOB_STARTED — Job started execution.
- SUBMIT_JOB — A job submission request was made.

Logging

- CRITICAL_LOG_AVAILABLE — Critical log records are available.
- DEBUG_LOG_AVAILABLE — Debug log records are available.
- ERROR_LOG_AVAILABLE — Error log records are available.
- EXCEPTION_LOG_AVAILABLE — Exception log records are available.
- INFO_LOG_AVAILABLE — Info log records are available.
- WARNING_LOG_AVAILABLE — Warning log records are available.

Run Lifecycle

- ABOUT_TO_END_RUN — Pre-notification: run about to end.
- ABOUT_TO_START_RUN — Pre-notification: run about to start.
- CHECK_END_RUN_READINESS — System coordination event.
- END_RUN — Current run ends on this process.
- PRE_RUN_RESULT_AVAILABLE — Pre-run results prepared and available.
- START_RUN — A new run begins on this process.

System & Resources

- `AUTHORIZE_COMMAND_CHECK` — Check authorization for admin command.
- `FATAL_SYSTEM_ERROR` — Fatal system-level error occurred.
- `SWAP_IN` — Bring resource(s) into active use.
- `SWAP_OUT` — Release/park resource(s).
- `SYSTEM_BOOTSTRAP` — System bootstrapping (initialize core services).
- `SYSTEM_END` — System shutting down.
- `SYSTEM_START` — System services starting up.

Task Dispatch & Results

- `ABORT_TASK` — System coordination event.
- `TASK_ASSIGNMENT_SENT` — Server sent a task assignment.
- `TASK_RESULT_RECEIVED` — Server received a task result.

Timing Hooks — General

- `AFTER_CHECK_CLIENT_RESOURCES` — Hook fired after check client resources.
- `AFTER_CLIENT_HEARTBEAT` — Hook fired after client heartbeat.
- `AFTER_CLIENT_REGISTER` — Hook fired after client register.
- `AFTER_JOB_LAUNCH` — Hook fired after job launch.
- `BEFORE_BUILD_COMPONENT` — Hook fired before build component.
- `BEFORE_CHECK_CLIENT_RESOURCES` — Hook fired before check client resources.
- `BEFORE_CHECK_RESOURCE_MANAGER` — Hook fired before check resource manager.
- `BEFORE_CLIENT_HEARTBEAT` — Hook fired before client heartbeat.
- `BEFORE_CLIENT_REGISTER` — Hook fired before client register.
- `BEFORE_JOB_LAUNCH` — Hook fired before job launch.
- `BEFORE_SEND_ADMIN_COMMAND` — Hook fired before send admin command.

Timing Hooks — Processing

- `AFTER_PROCESS_SUBMISSION` — Hook fired after process submission.
- `BEFORE_PROCESS_SUBMISSION` — Hook fired before process submission.

Timing Hooks — Task

- `AFTER_PROCESS_RESULT_OF_UNKNOWN_TASK` — Hook fired after process result of unknown task.
- `AFTER_PROCESS_TASK_REQUEST` — Hook fired after process task request.
- `AFTER_PULL_TASK` — Hook fired after pull task.
- `AFTER_SEND_TASK_RESULT` — Hook fired after send task result.
- `AFTER_TASK_DATA_FILTER` — Hook fired after task data filter.
- `AFTER_TASK_EXECUTION` — Hook fired after task execution.
- `AFTER_TASK_RESULT_FILTER` — Hook fired after task result filter.
- `BEFORE_PROCESS_RESULT_OF_UNKNOWN_TASK` — Hook fired before process result of unknown task.

- BEFORE_PROCESS_TASK_REQUEST — Hook fired before process task request.
- BEFORE_PULL_TASK — Hook fired before pull task.
- BEFORE_SEND_TASK_RESULT — Hook fired before send task result.
- BEFORE_TASK_DATA_FILTER — Hook fired before task data filter.
- BEFORE_TASK_EXECUTION — Hook fired before task execution.
- BEFORE_TASK_RESULT_FILTER — Hook fired before task result filter.

Workflow Lifecycle

- END_WORKFLOW — Workflow lifecycle ended.
- START_WORKFLOW — Workflow lifecycle started.

Application Events — AppEventType

Aggregation

- AFTER_AGGREGATION — Application hook after aggregation.
- BEFORE_AGGREGATION — Application hook before aggregation.
- END_AGGREGATION — Aggregation step ended.

Model IO & Best-Model

- AFTER_LEARNABLE_PERSIST — Application hook after learnable persist.
- AFTER_SEND_BEST_MODEL — Application hook after send best model.
- AFTER_SHAREABLE_TO_LEARNABLE — Application hook after shareable to learnable.
- BEFORE_LEARNABLE_PERSIST — Application hook before learnable persist.
- BEFORE_SEND_BEST_MODEL — Application hook before send best model.
- BEFORE_SHAREABLE_TO_LEARNABLE — Application hook before shareable to learnable.
- GLOBAL_BEST_MODEL_AVAILABLE — New global-best model available.
- GLOBAL_WEIGHTS_UPDATED — Global model/weights updated.
- INITIAL_MODEL_LOADED — Initial model loaded from persister.
- LOCAL_BEST_MODEL_AVAILABLE — Site has a new local-best model.
- RECEIVE_BEST_MODEL — Best-model payload received.
- RECEIVE_VALIDATION_MODEL — Site received a model to validate.
- SEND_MODEL_FOR_VALIDATION — Model sent to validators.
- SERVER_RECEIVE_BEST_MODEL — Server received a best-model submission.
- SUBMIT_LOCAL_BEST_MODEL — Site submitted local-best model to server.

Other

- CROSS_VAL_INIT — Initialize cross-site validation phase.

Rounds

- ROUND_DONE — Federated round completed.
- ROUND_STARTED — Federated round has started.

Send/Receive/Submit

- RECEIVE_CONTRIBUTION — Aggregator received a client contribution.

Timing Hooks — General

- AFTER_CONTRIBUTION_ACCEPT — Application hook after contribution accept.
- AFTER_INITIALIZE — Application hook after initialize.
- AFTER_VALIDATE_MODEL — Application hook after validate model.
- BEFORE_CONTRIBUTION_ACCEPT — Application hook before contribution accept.
- BEFORE_INITIALIZE — Application hook before initialize.
- BEFORE_VALIDATE_MODEL — Application hook before validate model.

Training

- AFTER_TRAIN — Application hook after train.

- BEFORE_TRAIN — Application hook before train.
- BEFORE_TRAIN_TASK — Application hook before train task.
- TRAINING_FINISHED — Training phase finished at a site.
- TRAINING_STARTED — Training phase started at a site.
- TRAIN_DONE — Site-side training task finished.

Validation

- AFTER_GET_VALIDATION_MODELS — Application hook after get validation models.
- AFTER_SUBMIT_VALIDATION_RESULTS — Application hook after submit validation results.
- BEFORE_GET_VALIDATION_MODELS — Application hook before get validation models.
- BEFORE_SUBMIT_VALIDATION_RESULTS — Application hook before submit validation results.
- RECEIVE_VALIDATION_RESULTS — Validation results received.
- SEND_VALIDATION_RESULTS — Validation results sent.
- VALIDATION_RESULT_RECEIVED — Workflow/training/validation coordination event.

2) Scatter-and-Gather: Event Sequence (1 client, 1 round)

- Run & workflow start on server (ABOUT_TO_START_RUN → START_RUN → START_WORKFLOW); client emits START_RUN.
- Server loads initial model from the persistor and fires INITIAL_MODEL_LOADED.
- Round begins with ROUND_STARTED on the server.
- Client pulls work (BEFORE_PULL_TASK → request → AFTER_PULL_TASK).
- Server prepares and assigns train task (BEFORE_PROCESS_TASK_REQUEST → BEFORE_TRAIN_TASK → [optional outgoing filters] → TASK_ASSIGNMENT_SENT → AFTER_PROCESS_TASK_REQUEST), embedding the current global model in the Shareable.
- Client applies incoming filters (optional), executes the task (BEFORE_TASK_EXECUTION → TRAINING_STARTED → TRAIN_DONE → AFTER_TASK_EXECUTION).
- Client prepares and sends result (BEFORE_TASK_RESULT_FILTER → AFTER_TASK_RESULT_FILTER → BEFORE_SEND_TASK_RESULT → send → AFTER_SEND_TASK_RESULT).
- Server receives and processes submission (TASK_RESULT_RECEIVED → [optional incoming filters] → BEFORE_PROCESS_SUBMISSION → BEFORE_CONTRIBUTION_ACCEPT → AFTER_CONTRIBUTION_ACCEPT → AFTER_PROCESS_SUBMISSION).
- Server aggregates and updates global (BEFORE_AGGREGATION → AFTER_AGGREGATION → GLOBAL_WEIGHTS_UPDATED); optionally persists (BEFORE_LEARNABLE_PERSIST → AFTER_LEARNABLE_PERSIST).
- Round ends (ROUND_DONE). For a 1-round job: CHECK_END_RUN_READINESS → END_WORKFLOW → END_RUN on server; client emits END_RUN.

2.a) Timing Windows & Formulas

Client compute (strict; includes filters): AFTER_TASK_RESULT_FILTER – BEFORE_TASK_DATA_FILTER

Client processing (broad; excludes wire): BEFORE_SEND_TASK_RESULT – AFTER_PULL_TASK

Server compute per round:

- Dispatch/build: AFTER_PROCESS_TASK_REQUEST – BEFORE_PROCESS_TASK_REQUEST
- Accept/process: AFTER_PROCESS_SUBMISSION – TASK_RESULT_RECEIVED
- Aggregation: AFTER_AGGREGATION – BEFORE_AGGREGATION
- Total server compute: $\Delta_{\text{dispatch}} + \Delta_{\text{accept}} + \Delta_{\text{agg}}$

Communication (RTT-like, measured at client):

- Downstream S→C: AFTER_PULL_TASK – BEFORE_PULL_TASK
- Upstream C→S: AFTER_SEND_TASK_RESULT – BEFORE_SEND_TASK_RESULT

Optional pure wire (if clocks synced or timestamps embedded):

- Upstream wire $\approx$ TASK_RESULT_RECEIVED@server – BEFORE_SEND_TASK_RESULT@client
- Downstream wire $\approx$ AFTER_PULL_TASK@client – send_ready@server ($\approx$ AFTER_PROCESS_TASK_REQUEST minus any server outgoing-filter time)

3) Diagram

Please refer to the attached diagram in the **docs/** directory:

- docs/sg_sequence_with_metrics.png (rendered diagram)
- docs/NVFLARE_SG_Sequence.mmd and docs/NVFLARE_SG_Sequence_with_Metrics.mmd (Mermaid sources)